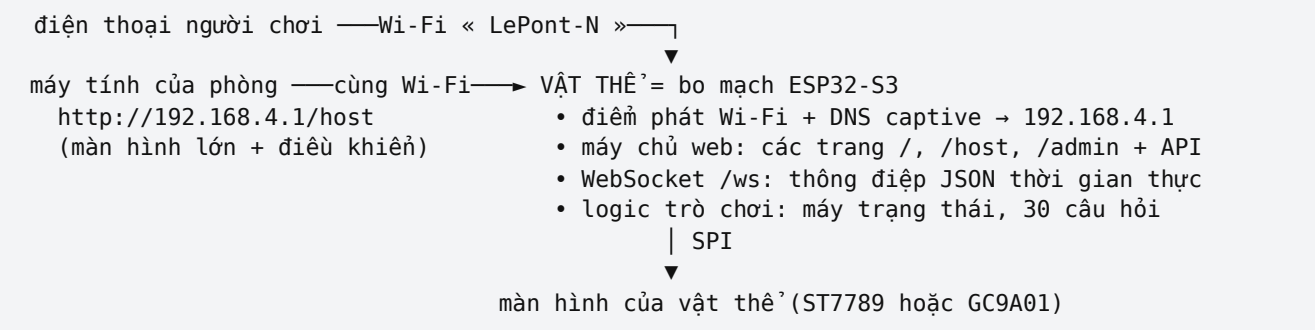
Le Pont · Cầu — Hồ sơ kỹ thuật từng bước

27/10/2026 · Yassine Bruneau, Makerspace của École des Ponts – IP Paris · workshop PFIEV × DUT Maker Innovation Space

Hồ sơ này phân rã, theo thứ tự, các công đoạn phát triển phần mềm đưa mã nguồn đến một nguyên mẫu chạy được trong tay sinh viên. Mỗi bước nêu mục tiêu, các thao tác cần làm, các tệp liên quan, bài kiểm tra xác nhận và những cái bẫy đã biết. Hồ sơ dựa trên phần mềm nền LePont v1 (bàn giao ngày 27/10) và là tài liệu tham chiếu cho giảng viên và hai sinh viên trợ giảng.

1. Kiến trúc trong một trang

Tất cả nằm trong một bo mạch ESP32-S3 giá dưới 10 €, làm năm việc cùng lúc: tạo mạng Wi-Fi, chuyển hướng điện thoại về trang của nó (cổng captive), phục vụ các trang web, trao đổi thông điệp thời gian thực với từng điện thoại và màn hình lớn (WebSocket) và chạy logic trò chơi. Điện thoại không cài gì cả: chúng nhận một trang HTML. Máy tính của phòng mở trang « màn hình lớn », đồng thời là bộ điều khiển.



Điện thoại và máy tính chỉ nói chuyện với vật thể, qua Wi-Fi riêng của nó; không có gì đi qua internet.

Lớp	Vai trò	Công nghệ	Tệp
Thiết lập	số nhóm, màn hình, chân cắm, các đội, thời lượng	hằng số C++	`config.h`
Mạng	điểm phát Wi-Fi `LePont-N`, địa chỉ 192.168.4.1, cổng captive	`WiFi.softAP`, `DNSServer`	`LePont.ino` (`setup`, `setupServer`)
Máy chủ web	các trang `/`, `/host`, `/admin`; API `/api/state`, `/api/questions`	ESPAsyncWebServer (nhánh ESP32Async) + AsyncTCP	`LePont.ino` (`setupServer`)
Thời gian thực	thông điệp JSON giữa vật thể, điện thoại và màn hình lớn	AsyncWebSocket, ArduinoJson 7	`LePont.ino` (`onWsEvent`, `onWsMessage`, `broadcast`)
Trò chơi	sảnh chờ, câu hỏi, đồng hồ, điểm, nhịp cầu, kết thúc ván	máy trạng thái viết bằng C++	`LePont.ino` (`startQuestion`, `reveal`, `endGame`, `resetGame`)
Dữ liệu	30 câu hỏi song ngữ nhúng sẵn, nạp lại nóng được	JSON trong PROGMEM	`questions.h`, `gen_questions_h.py`
Trang web	giao diện người chơi, màn hình lớn, quản trị	HTML + JavaScript không phụ thuộc thư viện, nhúng trong PROGMEM	`web/*.html` → `web_*.h` qua `gen_web_h.py`
Màn hình của vật thể	trạng thái trò chơi trên màn hình riêng của vật thể	GFX Library for Arduino, bus SPI	`display.h`
Kiểm thử	mô phỏng vật thể và kiểm thử đầu-cuối không cần phần cứng	Python (websockets) + Playwright	`test/mock_server.py`, `test/e2e.py`

2. Đường găng và tình trạng ngày 27/10

Mã của các bước 2 đến 7 đã viết xong và được thử không cần phần cứng (mô phỏng vật thể + kiểm thử đầu-cuối tự động, « E2E OK »); mã chưa được biên dịch bằng chuỗi công cụ Arduino thật. Nguyên mẫu được coi

là chạy được khi bước 9 được xác nhận: một ván chơi đầy đủ trên từng vật thể trong 7 vật thể. Còn khoảng 5 giờ làm việc từ lúc cài máy đến mốc đó, kể cả màn hình.

Bước	Sản phẩm	Tình trạng	Còn phải làm (thời lượng)
1. Môi trường	Arduino IDE + hỗ trợ ESP32 + thư viện	đang làm (máy tính sửa xong ngày 26)	cài đặt, thử WiFiScan (30 phút)
2. Wi-Fi và cổng captive	thấy mạng `LePont-N`, trang tự mở	đã viết mã	biên dịch lần đầu, thử Android + iPhone (45 phút)
3. Máy chủ web	các trang được phục vụ từ vật thể	đã viết mã	mở `/`, `/host`, `/admin` (10 phút)
4. WebSocket	thông điệp thời gian thực	đã viết mã, thử trên mô phỏng	kiểm tra với 2 điện thoại (15 phút)
5. Logic trò chơi	ván đầy đủ từ sảnh chờ → kết thúc	đã viết mã, E2E OK	ván thật với 3 hoặc 4 điện thoại (20 phút)
6. Câu hỏi	30 câu hỏi Pháp/Việt nhúng sẵn	xong	câu hỏi của các nhóm (buổi 1)
7. Trang web	người chơi, màn hình lớn, admin	xong, thử bằng Playwright	độ dễ đọc trên điện thoại thật và máy chiếu (10 phút)
8. Màn hình của vật thể	hiển thị trạng thái trò chơi	đã viết mã (ST7789 và GC9A01)	nối dây, `DISPLAY_TYPE 1`, thử (1 giờ)
9. Tích hợp	7 vật thể đã nạp, 7 kênh, pin dự phòng	kế hoạch đã viết, mã QR in được	nạp 7 bo, thử tải với 8 đến 10 điện thoại (1 giờ 30)
10. V2	màn hình tròn cảm ứng, biến thể trò chơi	chưa làm	2/11, khi nhận màn hình (nửa ngày)

Thứ tự bắt buộc: 1 → 2 → 3 → 4 → 5, mỗi bước được kiểm tra trước khi sang bước sau, vì một lỗi ở bước 5 có thể đến từ bất kỳ bước nào trước đó. Các bước 6, 7 và 8 độc lập và có thể giao cho các trợ giảng làm song song. Lần biên dịch đầu tiên có lẽ sẽ sinh vài lỗi về phiên bản thư viện: sửa chúng thuộc bước 2, dự kiến 30 phút.

Bước 1 — Môi trường phát triển

Mục tiêu. Một máy tính biên dịch được dự án và nạp được lên ESP32-S3. Mỗi nhóm một máy là đủ; máy của giảng viên là máy tham chiếu.

Thao tác.

1. Cài Arduino IDE 2 (Windows, macOS hoặc Linux).
2. File → Preferences → Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`, rồi Boards Manager → cài **esp32 by Espressif Systems** 3.x (khoảng 1 GB, phần lâu nhất).
3. Library Manager: **ESP Async WebServer** (ESP32Async, đồng ý cài kèm **Async TCP**), **ArduinoJson** 7.x, **GFX Library for Arduino**.
4. Cắm bo bằng cáp USB-C **truyền dữ liệu** (cổng COM/UART nếu bo có hai cổng). Không thấy cổng: cài driver **CH343** hoặc **CP210x** tùy con chip cạnh cổng cắm.
5. Menu Tools: ESP32S3 Dev Module; USB CDC On Boot: Disabled (cắm cổng COM; chỉ chọn Enabled khi cắm cổng USB); Flash Size: 16 MB; PSRAM: OPI PSRAM; Partition Scheme: Huge APP (3 MB No OTA / 1 MB SPIFFS); Upload Speed: 921600.

Tệp. `HUONG_DAN_CAI_DAT.md` (phát cho các nhóm trước buổi học).

Xác nhận. File → Examples → WiFi → WiFiScan đã nạp; Serial Monitor ở 115200 baud liệt kê các mạng Wi-Fi. Nếu bước này đạt, toàn bộ chuỗi máy tính → bo mạch hoạt động.

Bẫy. Cáp chỉ sạc (không có cổng); « Failed to connect »: giữ BOOT, bấm RESET, thả BOOT, nạp lại; « sketch too big »: sai phân vùng; khởi động lặp vô tận: PSRAM đặt sai.

Bước 2 — Điểm phát Wi-Fi và cổng captive

Mục tiêu. Điện thoại thấy mạng `LePont-N`, kết nối không cần mật khẩu và trang trò chơi tự mở.

Thao tác.

1. Trong `setup()`: `WiFi.mode(WIFI_AP)`, `WiFi.softAPConfig(192.168.4.1, 192.168.4.1, 255.255.255.0)`, rồi `WiFi.softAP(ssid, "", kênh, 0, 10)`. SSID được ghép từ `GROUP_ID`, kênh đọc từ `WIFI_CHANNELS` (1, 6 và 11 phân bố cho 7 nhóm), tối đa 10 kết nối.
2. Khởi động `DNSServer` trên cổng 53 với ký tự đại diện `*`: mọi truy vấn tên đều trả về 192.168.4.1.
3. Trả lời các yêu cầu kiểm tra kết nối của hệ điều hành bằng chuyển hướng 302 về `http://192.168.4.1/`: Android `/generate_204`, iOS và macOS `/hotspot-detect.html`, Windows `/connecttest.txt` và `/ncsi.txt`. Route `onNotFound` chuyển hướng tương tự.
4. Trong `loop()`: `dns.processNextRequest()` ở mỗi vòng lặp, không dùng `delay()`.

Tệp. `LePont.ino` (`setup`, `setupServer`), `config.h` (`GROUP_ID`, `WIFI_CHANNELS`, `WIFI_PASSWORD`, `MAX_PLAYERS`).

Xác nhận. Serial Monitor hiện `[wifi] LePont-N … OK`. Android đề nghị « Đăng nhập mạng » và mở trang; iPhone mở cửa sổ captive. Trong mọi trường hợp, gõ tay `http://192.168.4.1` đều hoạt động.

Bẫy. Cửa sổ captive của iPhone là một trình duyệt thu nhỏ không ổn định với WebSocket: yêu cầu mở Safari tại địa chỉ đó. Điểm phát chỉ ở băng tần 2,4 GHz. Quá 8 đến 10 điện thoại, bo mạch từ chối kết nối (`full`). Điện thoại « không có internet » có thể tự chuyển về 4G: giữ mạng bằng tay.

Bước 3 — Máy chủ web và các trang nhúng

Mục tiêu. Vật thể phục vụ ba trang và API của nó mà không cần thẻ SD, không internet và không cài gì trên điện thoại.

Thao tác.

1. Các trang được viết bằng HTML và JavaScript trong `web/` (`player.html`, `host.html`, `admin.html`), không dùng thư viện ngoài.
2. `python3 gen_web_h.py` chuyển mỗi trang thành chuỗi thô `PROGMEM` (`R"html(…)html"`) trong `web_player.h`, `web_host.h`, `web_admin.h`: các trang nằm trong flash, không nằm trong RAM.
3. Các route khai báo trong `setupServer()`: `/` → trang người chơi; `/host` → màn hình lớn; `/admin` → câu hỏi; `/api/state` → trạng thái dạng JSON; `/api/questions` dạng GET (bộ hiện tại) và POST (nạp nóng).
4. Phản hồi gửi kèm `Content-Type: text/html; charset=utf-8` để dấu tiếng Pháp và tiếng Việt hiển thị đúng.
5. Sau mỗi lần sửa trang: chạy lại `gen_web_h.py`, rồi nạp lại.

Tệp. `web/*.html`, `gen_web_h.py`, `web_*.h`, `LePont.ino` (`setupServer`).

Xác nhận. Ba trang mở được từ điện thoại và từ máy tính; `http://192.168.4.1/api/state` trả về JSON đọc được.

Bẫy. Quên tạo lại các tệp `.h` (trang hiển thị là bản cũ); tệp lưu không phải UTF-8; trang nào tải font hoặc script từ internet sẽ trắng trơn, vì vật thể không nối internet.

Bước 4 — WebSocket và giao thức thông điệp

Mục tiêu. Mỗi điện thoại và màn hình lớn nhận trạng thái trò chơi ngay khi nó thay đổi, không cần tải lại trang.

Thao tác.

1. `AsyncWebSocket ws("/ws")` gắn vào máy chủ; `onWsEvent` xử lý ba trường hợp: kết nối (gửi `config`), ngắt kết nối (người chơi trở thành không hoạt động), dữ liệu (chuyển cho `onWsMessage`).
2. Một thông điệp = một đối tượng JSON có trường `t` (loại). ArduinoJson đọc ở phía bo mạch, `JSON.parse` ở phía trang.
3. Hai hàm gửi: `broadcast(doc)` = `ws.textAll(…)` tới tất cả; `sendTo(client, doc)` tới một người.
4. `ws.cleanupClients()` mỗi giây trong `loop()` để giải phóng các kết nối đã chết.
5. Phía trang: `new WebSocket('ws://' + location.host + '/ws')`, tự kết nối lại 2 giây sau khi mất kết nối, gửi lại `hello` ở mỗi lần kết nối lại để nhận trạng thái hiện tại.

Chiều	Loại `t`	Nội dung	Khi nào
điện thoại → vật thể	`join`	`name`, `team`	vào ván, hoặc quay lại với cùng tên
điện thoại → vật thể	`answer`	`a` (0 đến 3)	mỗi câu hỏi chỉ một câu trả lời

Chiều	Loại `t`	Nội dung	Khi nào
màn hình lớn → vật thể	`hello`	`role: "host"`	mở `/host`
màn hình lớn → vật thể	`host`	`cmd: start / next / reveal / reset`	các nút điều khiển
vật thể → tất cả	`config`	đội, SSID, số câu hỏi, thời lượng	ở mỗi kết nối
vật thể → tất cả	`lobby`	người chơi, điểm	mỗi lần vào hoặc ra
vật thể → tất cả	`question`	`i`, `n`, `cat`, `fr`, `vi`, `o[4]`, `sec`	bắt đầu câu hỏi
vật thể → tất cả	`tick`	`left`, `answers`	mỗi giây
vật thể → tất cả	`reveal`	`r`, `pfr`, `pvi`, `won`, `spans`, `scores`	hết câu hỏi
vật thể → tất cả	`end`, `reset`	đội thắng, nhịp cầu, điểm	hết ván, đặt lại
vật thể → một người chơi	`joined`, `you`, `full`	định danh; `ok`, `pts`; từ chối	sau `join`, sau lộ đáp án, vật thể đầy

Tệp. `LePont.ino` (`onWsEvent`, `onWsMessage`, `broadcast`, `sendTo`), `web/player.html`, `web/host.html`.

Xác nhận. Console trình duyệt (F12) trên `/host`: các thông điệp `config` rồi `lobby` đến; một `join` từ điện thoại làm người chơi hiện trên màn hình lớn trong chưa đầy một giây.

Bẫy. Gửi mà không kiểm tra `client->canSend()`; thông điệp quá dài (giữ JSON ngắn, không bao giờ đưa bộ câu hỏi vào một thông điệp); mỗi kết nối đang mở tiêu tốn RAM, vì thế phải bật PSRAM và giới hạn số người chơi.

Bước 5 — Logic trò chơi

Mục tiêu. Một ván chơi đầy đủ, dự đoán được, chơi lại được mà không cần khởi động lại vật thể.

```
 Sảnh chờ ──Bắt đầu──► Câu hỏi ──tất cả đã trả lời hoặc hết 20 s──► Lộ đáp án ──Câu tiếp (câu 2 đến 10)──► Câu hỏi …
                                                                                  └──sau câu thứ 10──►
Kết thúc ──Chơi lại (giữ người chơi)──► Sảnh chờ
 Reset đưa về sảnh chờ từ bất kỳ trạng thái nào và xóa người chơi cùng điểm số.
```

Vòng lặp câu hỏi → lộ đáp án chạy mười lần; chỉ màn hình lớn mới đẩy ván chơi đi tiếp.

Thao tác.

1. Máy trạng thái: một biến `state` và bốn hàm duy nhất được phép thay đổi nó: `startQuestion()`, `reveal()`, `endGame()`, `resetGame()`. Mọi sự kiện (thông điệp WebSocket, đồng hồ) đều đi qua chúng.
2. Người chơi: mảng `Player {id, name, team, score, answer, answerMs, active}` giới hạn bởi `MAX_PLAYERS`. Người chơi quay lại với cùng tên sẽ lấy lại chỗ và điểm của mình.
3. Câu hỏi: rút không hoàn lại `QUESTIONS_PER_GAME` (10) câu bằng `shuffleOrder()`; đồng hồ `QUESTION_SECONDS` (20 giây) duy trì trong `loop()` bằng `millis()`; mỗi giây một `tick` kèm số câu trả lời đã nhận.
4. Lộ đáp án: kích hoạt khi tất cả đã trả lời, khi hết giờ, hoặc bằng nút Révéler. Điểm = 100 + thưởng tốc độ tối đa 50; một nhịp cầu được đặt nếu ít nhất một nửa người trả lời đúng; điểm cộng dồn theo đội.
5. Kết thúc: sau câu thứ 10, `end` (đội thắng, nhịp cầu, điểm). Chơi lại đưa về sảnh chờ và giữ người chơi; Reset xóa tất cả.

Tệp. `LePont.ino` (`startQuestion`, `reveal`, `endGame`, `resetGame`, `loop`), `config.h` (thời lượng, số câu hỏi, đội).

Xác nhận. Không cần phần cứng: `python3 test/e2e.py` khởi động mô phỏng và chơi một ván trong ba trình duyệt (màn hình lớn + hai người chơi); kết quả mong đợi `E2E OK`. Trên phần cứng: một ván với ba điện thoại, trong đó một chiếc tắt Wi-Fi giữa chừng rồi quay lại.

Bẫy. Không bao giờ dùng `delay()`: máy chủ bất đồng bộ sẽ đứng. Không bao giờ sửa `state` trực tiếp từ một callback WebSocket. Thiếu `tick` gần như luôn do `loop()` bị phần hiển thị chặn.

Bước 6 — Bộ câu hỏi

Mục tiêu. Câu hỏi song ngữ mà các nhóm sửa được mà không chạm vào mã C++.

Thao tác.

1. Nguồn duy nhất: `PONT_questions_FR_VI.csv` (các cột `id`, `categorie`, `question_fr`, `question_vi`, `option_A` đến `option_D`, `reponse`, `cible_estimation`, `pont_fr`, `pont_vi`, `source`).
2. `build_questions.py` tạo JSON; `gen_questions_h.py` nhúng vào `questions.h` dưới dạng `QUESTIONS_JSON` trong PROGMEM. Mỗi câu hỏi trở thành `{cat, fr, vi, o[4], r, pfr, pvi, num?}`; `r` là chỉ số của phương án đúng, từ 0 đến 3.
3. Khi khởi động, `loadQuestions()` đọc JSON bằng ArduinoJson vào `qdoc`, đặt trong PSRAM.
4. Nạp nóng: trang `/admin` gửi JSON mới bằng POST tới `/api/questions`; nó thay thế bộ câu hỏi trong RAM cho đến lần khởi động lại tiếp theo. Để lưu vĩnh viễn: tạo lại `questions.h` và nạp lại.
5. Buổi 1: mỗi nhóm soạn 10 câu hỏi theo định dạng này (mỗi câu một dòng trong CSV, hoặc trực tiếp bằng JSON trong `/admin`).

Tệp. `PONT_questions_FR_VI.csv`, `build_questions.py`, `gen_questions_h.py`, `questions.h`, `web/admin.html`.

Xác nhận. `http://192.168.4.1/api/questions` trả về 30 đối tượng; hai ván liên tiếp không hỏi theo cùng thứ tự; dán JSON vào `/admin` trả về « N câu hỏi đã nạp ».

Bẫy. Thừa một dấu phẩy hoặc dấu nháy làm JSON không hợp lệ: `/admin` báo lỗi, bộ cũ vẫn giữ nguyên. Câu hỏi ước lượng (`num`) có trong bộ nhưng V1 bỏ qua; chúng là bài tập cho buổi 2. Dấu nháy đơn thẳng trong câu hỏi tiếng Pháp không gây vấn đề; dấu nháy kép thẳng `"` phải được thoát hoặc thay bằng dấu ngoặc kép kiểu Pháp.

Bước 7 — Trang người chơi, màn hình lớn, quản trị

Mục tiêu. Giao diện đọc được bằng ngón cái trên điện thoại và từ xa ba mét trên máy chiếu, không tải tài nguyên nào từ internet.

Thao tác.

1. `player.html`: màn hình chào (tên, chọn đội), phòng chờ, câu hỏi với bốn nút lớn, kết quả và điểm, kết thúc ván; tự kết nối lại với cùng tên.
2. `host.html`: sảnh chờ (Wi-Fi, địa chỉ, người chơi), câu hỏi (đồng hồ, thanh thời gian, văn bản tiếng Pháp rồi tiếng Việt, bốn phương án, lộ đáp án song ngữ), cây cầu SVG với các nhịp được đặt luân phiên từ hai bờ, điểm theo đội, các nút Reset / Révéler / Lancer → Suivante → Rejouer.
3. `admin.html`: ô văn bản JSON điền sẵn bộ hiện tại, nút nạp, thông báo phản hồi.
4. Thử không cần ESP32: `python3 test/mock_server.py` giả lập vật thể (HTTP trên 8080, WebSocket trên 8081, cùng giao thức, ván ngắn 3 câu, 5 giây); `python3 test/e2e.py` chơi một ván đầy đủ bằng Playwright và lưu ảnh chụp vào `test/shots/`.
5. Sau khi sửa: `gen_web_h.py`, rồi nạp lại. Trợ giảng có thể sửa các trang trên máy của mình và thử với mô phỏng mà không chiếm một vật thể.

Tệp. `web/player.html`, `web/host.html`, `web/admin.html`, `test/mock_server.py`, `test/e2e.py`, `test/q.json`.

Xác nhận. `E2E OK` trên mô phỏng; trên phần cứng, một người chơi đọc được câu hỏi từ cuối phòng trên máy chiếu và trả lời bằng ngón cái không cần phóng to.

Bẫy. Địa chỉ WebSocket phải giữ là `location.host` (không bao giờ ghi cứng, nếu không mô phỏng và vật thể sẽ lệch nhau). Màn hình lớn ở 1280×720 từng bị tràn dưới các nút: đã giảm cỡ chữ và thêm lề dưới, cần xem lại nếu đổi nội dung chữ.

Bước 8 — Màn hình của vật thể

Mục tiêu. Vật thể tự hiển thị trạng thái trò chơi: tên mạng và số người đang chờ, vòng đếm ngược, kết quả, điểm, đội thắng.

Thao tác.

1. Nối màn hình SPI vào ESP32-S3 bằng bảy dây Dupont cái-cái. Chân cắm thay đổi được trong `config.h`.
2. Kiểm tra dây nối bằng một ví dụ của GFX Library for Arduino (HelloWorld) cấu hình cùng bus và cùng driver, trước khi đụng tới trò chơi.
3. Trong `config.h`, đặt `DISPLAY_TYPE` thành `1` (ST7789 1,3" vuông, 240×240) hoặc `2` (GC9A01 1,28" tròn, 240×240), rồi nạp lại.

4. `display.h` cung cấp sáu hàm được trò chơi gọi: `displayBegin`, `displayLobby`, `displayQuestion`, `displayReveal`, `displayScores`, `displayEnd`. Với `DISPLAY_TYPE 0`, chúng ghi ra cổng serial; nếu không, chúng vẽ bằng `Arduino_ESP32SPI` + `Arduino_ST7789` hoặc `Arduino_GC9A01`.
5. Tùy biến màn hình = chỉ sửa `display.h`; `LePont.ino` không biết gì về màn hình. Đây là tệp giao cho các nhóm để vibe coding ở buổi 2.

Dây màn hình	Chân ESP32-S3	Hằng số
VCC	3V3	—
GND	GND	—
SCL (xung SPI)	GPIO 12	`PIN_SCK`
SDA (dữ liệu SPI)	GPIO 11	`PIN_MOSI`
CS	GPIO 10	`PIN_CS`
DC	GPIO 9	`PIN_DC`
RES	GPIO 8	`PIN_RST`
BLK (đèn nền)	GPIO 7	`PIN_BL`

Tệp. `display.h`, `config.h`.

Xác nhận. Sảnh chờ hiện SSID và số người chơi; trong câu hỏi, vòng tròn cạn dần trong 20 giây; khi lộ đáp án, màu của đội dẫn đầu xuất hiện; `tick` trên màn hình lớn vẫn đều (hiển thị không làm chậm trò chơi).

Bẫy. Màn hình trắng: bus hoặc RES; hình bị lệch: offset (80 dòng trên ST7789 1,3"); màu bị đảo: tham số IPS của nhà sản xuất; chỉ cấp nguồn 3,3 V, không bao giờ 5 V vào VCC; vẽ nặng trong `loop()` làm đồng hồ đứng, chỉ vẽ khi trạng thái thay đổi.

Bước 9 — Tích hợp và thử trong điều kiện thật

Mục tiêu. Bảy vật thể hoạt động cùng lúc trong cùng một phòng, mỗi vật thể với người chơi, màn hình lớn và pin dự phòng của mình.

Thao tác.

1. Nạp bảy bo chỉ thay `GROUP_ID` (1 đến 7): SSID `LePont-1` đến `LePont-7`, kênh 1 / 6 / 11 / 1 / 6 / 11 / 1. Dán nhãn số lên từng bo.
2. In tờ A4 mã QR Wi-Fi của bảy nhóm và một áp phích nối dây cho mỗi bàn.
3. Cấp nguồn mỗi vật thể bằng pin dự phòng (cáp sạc là đủ sau khi đã nạp); kiểm tra pin dự phòng không tự tắt vì dòng tiêu thụ thấp (một số loại ngắt dưới 100 mA: màn hình bật giúp tránh điều này).
4. Thử tải trên một vật thể: 8 đến 10 điện thoại, một ván đầy đủ, độ trễ cảm nhận ở `tick` và khi lộ đáp án.
5. Thử cả phòng: bảy vật thể bật, bảy máy tính mở `/host`, một câu hỏi được chơi đồng thời ở mọi bàn.
6. Thử độ bền: rút điện một vật thể giữa câu hỏi, cắm lại; người chơi phải tự kết nối lại và thấy lại sảnh chờ.
7. Vỏ hộp: mẫu tham số (đường kính màn hình, lỗ USB-C, chỗ đặt bo), lịch in trải trong 24 giờ cho một máy in và bảy nhóm.

Tệp. `config.h` (`GROUP_ID`, `WIFI_CHANNELS`), `QR_WiFi_7_groupes_A4.pdf`, mẫu vỏ hộp 3D.

Xác nhận. Một ván đầy đủ trên từng vật thể trong bảy vật thể mà giảng viên không phải can thiệp; đó là mốc « nguyên mẫu chạy được ».

Bẫy. Hai nhóm cạnh nhau cùng kênh sẽ gây nhiễu: xếp xa nhau các bàn 1-4-7, 2-5 và 3-6. Máy tính màn hình lớn đôi khi vẫn ở Wi-Fi của trường: chuyển sang `LePont-N` và kiểm tra địa chỉ 192.168.4.x. Điện thoại « quên » mạng sẽ quay về 4G: để tờ QR trong tầm tay.

Bước 10 — V2: màn hình tròn cảm ứng và các biến thể

Mục tiêu. Vật thể hoàn chỉnh: màn hình tròn, cảm ứng, vỏ hộp kín, và ván chơi điều khiển được từ chính vật thể.

Thao tác.

1. Nhận bảy màn hình GC9A01 1,28" ngày 2/11: thử từng cái với HelloWorld, rồi `DISPLAY_TYPE 2`; nối dây SPI giống V1.
2. Cảm ứng: bộ điều khiển CST816S của mô-đun giao tiếp qua I2C. Thêm chân của nó vào `config.h` (`PIN_TP_SDA`, `PIN_TP_SCL`, `PIN_TP_INT`, `PIN_TP_RST`), cài thư viện CST816S, đọc thao tác chạm trong

`loop()` không chặn. Chạm nhanh = Bắt đầu / Câu tiếp, chạm giữ = Lộ đáp án: vật thể trở thành bộ điều khiển của chính nó, máy tính chỉ còn dùng để hiển thị.

3. Vòng chơi « ước lượng »: loại câu hỏi mới `num`; ô nhập số trong `player.html`, hiển thị các ước lượng trong `host.html`, 150 điểm cho người gần nhất trong `reveal()`.
4. Bốn đội (`TEAM_COUNT 4`) và bảng xếp hạng cuối; hiển thị đội dẫn đầu trong `display.h`.
5. Vỏ hộp kín chứa pin dự phòng, công tắc nguồn dễ với tới.

Tệp. `config.h`, `display.h`, `LePont.ino` (`reveal`, `loop`), `web/player.html`, `web/host.html`.

Xác nhận. Một ván V2 đầy đủ điều khiển bằng ngón tay từ vật thể, không chạm vào máy tính; một vòng ước lượng được chơi và tính điểm đúng.

Bẫy. Xung đột chân giữa SPI và I2C trên một số mô-đun (RST dùng chung); đọc cảm ứng kiểu chặn trong `loop()` làm hỏng `tick`; phân vùng Huge APP là bắt buộc, firmware V2 vượt 1,5 MB.

Xử lý sự cố nhanh

Phản xạ đầu tiên luôn là Serial Monitor ở 115200 baud: vật thể kể lại nó đang làm gì (`[wifi]`, `[ws]`, `[game]`).

Triệu chứng	Nguyên nhân có thể	Xử lý
Không có cổng serial trong Tools	thiếu driver, cáp chỉ sạc	cài CH343 hoặc CP210x, đổi cáp
« Failed to connect to ESP32 »	bo không ở chế độ nạp	giữ BOOT, bấm RESET, thả BOOT, nạp lại
« Sketch too big »	sai phân vùng	Partition Scheme: Huge APP
Khởi động lặp vô tận	PSRAM đặt sai	OPI PSRAM
Serial Monitor chỉ hiện `entry 0x403c88b8`, không có gì từ trò chơi	Cổng COM với USB CDC On Boot: Enabled	USB CDC On Boot: Disabled, rồi Upload
Không thấy mạng `LePont-N`	treo khi khởi động, kênh	đọc Serial Monitor cho đến `[wifi] … OK`
Trang không tự mở	yêu cầu captive không bị chặn	gõ `http://192.168.4.1`; trên iPhone, mở Safari
Trang trắng hoặc bản cũ	`.h` chưa tạo lại, bộ nhớ đệm trình duyệt	`gen_web_h.py`, nạp lại, tải lại không dùng cache
Nút Bắt đầu không phản ứng	WebSocket đã đóng, máy tính ở Wi-Fi khác	console F12, kiểm tra máy tính đang ở `LePont-N`
Đồng hồ nhảy giây	`loop()` bị chặn (hiển thị, `delay`)	chỉ vẽ khi đổi trạng thái, bỏ mọi `delay()`
Một người chơi biến mất	điện thoại mất Wi-Fi	kết nối lại với cùng tên: giữ chỗ và điểm
`full` trên điện thoại	quá 10 kết nối	chia sang hai vật thể
Màn hình trắng hoặc đen	dây SPI, RES, đèn nền	kiểm tra lại bảy dây, thử HelloWorld
Hình lệch, màu đảo	offset hoặc IPS của nhà sản xuất	chỉnh trong `display.h`
Không nạp được câu hỏi	JSON không hợp lệ	đọc thông báo lỗi của `/admin`, sửa dòng đó

Thuật ngữ

Thuật ngữ	Trong một dòng
ESP32-S3	vi điều khiển Wi-Fi của Espressif; bản N16R8: 16 MB flash, 8 MB PSRAM
Firmware, nạp (flash)	chương trình đã biên dịch; ghi nó vào bo qua USB
SoftAP	bo tự tạo mạng Wi-Fi của riêng mình thay vì tham gia một mạng
Cổng captive	tự động chuyển hướng tới một trang ngay khi vào mạng
DNS	dịch tên thành địa chỉ; ở đây, mọi tên đều dẫn tới vật thể
HTTP, route	một địa chỉ (`/host`) và phản hồi mà máy chủ gắn với nó

Thuật ngữ	Trong một dòng
WebSocket	kết nối thường trực cho phép vật thể gửi mà không cần được hỏi
JSON	định dạng văn bản của thông điệp và câu hỏi, mọi ngôn ngữ lập trình đều đọc được
PROGMEM	dữ liệu lưu trong flash thay vì RAM
PSRAM	RAM ngoài bổ sung, không thể thiếu cho câu hỏi và các kết nối
Máy trạng thái	chương trình mô tả bằng các trạng thái và các sự kiện làm chúng thay đổi
SPI	bus nối tiếp tốc độ cao 4 dây điều khiển màn hình
I2C	bus nối tiếp 2 dây, dùng cho bộ điều khiển cảm ứng
Phân vùng	cách chia flash; Huge APP dành 3 MB cho chương trình
Vibe coding	lập trình bằng cách mô tả kết quả mong muốn cho AI, rồi thử và sửa